Dubins Paths for Waveguide Routing

20 min read • Nov 1, 2023 • Quentin Wach

Read the **PDF** .

What does the perfect waveguide look like? Traditional waveguide interconnects lack efficiency and optimization, leading to suboptimal photonic integrated circuits (PICs). Addressing key concerns like minimizing internal and radiation losses, reducing cross-talk, and avoiding unnecessary crossings, I explored the concept of finding the shortest path between two points in a PIC given a minimum bending radius. Dubins paths, already well known in robotics and control theory, emerged as a simple and highly practical solution.

photonic integrated circuits | design automation | robotics | physics

1 Introduction

I'll try to keep this short and not ramble too much. I just want to get this out there since I found it rather useful and can't believe it isn't yet standard.

When I started my work at the Fraunhofer HHI, it became quickly obvious that the typical waveguide interconnects provided by libraries are not well behaved nor optimal as to improve the performance and compactness of the photonic integrated circuit. Most commonly we have simple straights, circular arcs, all sorts of splines, as well as Euler-bends, in short: They are all simple analytical functions. But then there are also splines that optimize themselves. Yet all of these are either simple building blocks that leave most of the work to the designer who has to plan out and specify every route, or behave extremely poorly and lead to overly wavey results that may even cross each other.

So in my first two weeks there, I asked myself this question. We have to consider multiple things:

- As short as possible as to minimize internal losses.
- As straight and continuos as possible as to minimize radiation losses.
- As far away from other waveguides and structures as to minimize cross-talk.
- Does not cross other waveguides unless close to a 90° angle if absolutely necessary.

It became quickly obvious that trying to find an optimimum for all these design paramters is impossible unless we either weigh their importance using a complicated metric we have to justify, or make much harder contraints.

Since it is common to have a fixed minimum bending radius for waveguides, it makes sense to translate this here.

I then boiled all of the considerations down to the single, I believe, most important question: "What is the shortest path between the pin a and pin b given a minimum bending radius?" or rather what is the shortest path between two 2D-vectors $\vec{a}$ and $\vec{b}$ given a minimum bending radius?

As it turns out, the answer is rather simple and long established in the field of robotics / control theory. **Dubins paths!**

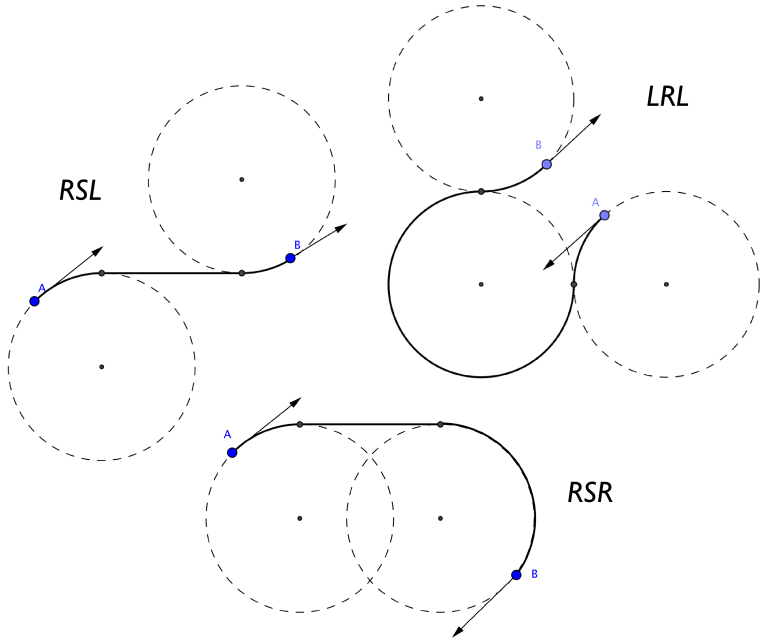
2 What are they?

Dubins paths are named after Lester Dubins, who introduced them in the 1950s[1]. They refer to the shortest paths that a vehicle can take from one point to another while constrained to move at a specific minimum turning radius. These paths are thus commonly studied in the field of motion planning for vehicles, particularly in robotics and aerospace engineering.

Now, to boil things down, these paths simply connect circular arcs with straights which leaves us with a couple of characteristic Dubins paths:

- **LRL** (Left-Right-Left)
- **RLR** (Right-Left-Right)
- **LSL** (Left-Straight-Left)
- **RSR** (Right-Straight-Right)
- **LSR** (Left-Straight-Right)
- **RSL** (Right-Straight-Left)

You can see three examples of the possible paths in the figure I adapted here[2].



3 Geometric Construction

Dubins did indeed prove that these trajectories are the shortest paths mathematically. But this can also understood quite intuitively. After all, on a flat surface, straights are the shortest path between any two points. Thus if the goal is to connect two points with a given orientation and under the restriction of a minimum bending radius, the way to find a shortest path is to optimize it such that we, ideally, get a straight line as long as possible.

A great overview and explanation of the synthesis of Dubins paths is given by David A. Anisi[3]. The geometric construction is quite intuitive and a nice toy problem to figure out on ones own but it can quickly explode into multiple pages of pen and paper calculations and diagrams. The whole process can be summarized in three steps:

1) From the given data, create the position vectors r and the velocity vectors V.

2) Find the center of the two circles at each end point, i.e. constructrc+andrc−by rotatingV, ±90◦.

3) From each of the two circles at the starting point, find a tangent connecting it to each of the two end point circles, such that it satisfies (4.11). Practically, the most convenient way to achieve this, is in a two phase manner; firstly we find all the candidates that are solutions to the non-linear equation system consisting of the two upper constraints in (4.11), upon which we check regularity and select the proper solution by means of the last constraint.

4 Code

There is an abundance of implementations of Dubins paths available on the internet[4].

```
#! /usr/bin/python
# -*- coding: utf-8 -*-
"""
Implementation of the basic Dubin's path algorithm.
Created by Quentin Wach.
October 19, 2023
"""
import math as m
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import nazca as nd
from nazca.interconnects import Interconnect

##########################################################################
# Main
##########################################################################

# finds the optimal path between two points using various planning methods
def general_planner(planner, alpha, beta, d):
    sa = m.sin(alpha)
    sb = m.sin(beta)
    ca = m.cos(alpha)
    cb = m.cos(beta)
    c_ab = m.cos(alpha - beta)
    mode = list(planner)

    planner_uc = planner.upper()

    if planner_uc == 'LSL':
        tmp0 = d + sa - sb
        p_squared = 2 + (d * d) - (2 * c_ab) + (2 * d * (sa - sb))
        if p_squared < 0:
            return None
        tmp1 = m.atan2((cb - ca), tmp0)
        t = mod_to_pi(-alpha + tmp1)
        p = m.sqrt(p_squared)
        q = mod_to_pi(beta - tmp1)
```

```
    elif planner_uc == 'RSR':
        tmp0 = d - sa + sb
        p_squared = 2 + (d * d) - (2 * c_ab) + (2 * d * (sb - sa))
        if p_squared < 0:
            return None
        tmp1 = m.atan2((ca - cb), tmp0)
        t = mod_to_pi(alpha - tmp1)
        p = m.sqrt(p_squared)
        q = mod_to_pi(-beta + tmp1)

    elif planner_uc == 'LSR':
        p_squared = -2 + (d * d) + (2 * c_ab) + (2 * d * (sa + sb))
        if p_squared < 0:
            return None
        p = m.sqrt(p_squared)
        tmp2 = m.atan2((-ca - cb), (d + sa + sb)) - m.atan2(-2.0, p)
        t = mod_to_pi(-alpha + tmp2)
        q = mod_to_pi(-mod_to_pi(beta) + tmp2)

    elif planner_uc == 'RSL':
        p_squared = (d * d) - 2 + (2 * c_ab) - (2 * d * (sa + sb))
        if p_squared < 0:
            return None
        p = m.sqrt(p_squared)
        tmp2 = m.atan2((ca + cb), (d - sa - sb)) - m.atan2(2.0, p)
        t = mod_to_pi(alpha - tmp2)
        q = mod_to_pi(beta - tmp2)

    elif planner_uc == 'RLR':
        tmp_rlr = (6.0 - d * d + 2.0 * c_ab + 2.0 * d * (sa - sb)) / 8.0
        if abs(tmp_rlr) > 1.0:
            return None

        p = mod_to_pi(2 * m.pi - m.acos(tmp_rlr))
        t = mod_to_pi(alpha - m.atan2(ca - cb, d - sa + sb) + mod_to_pi(p / 2.0
        q = mod_to_pi(alpha - beta - t + mod_to_pi(p))

    elif planner_uc == 'LRL':
        tmp_lrl = (6. - d * d + 2 * c_ab + 2 * d * (- sa + sb)) / 8.
        if abs(tmp_lrl) > 1:
            return None
        p = mod_to_pi(2 * m.pi - m.acos(tmp_lrl))
        t = mod_to_pi(-alpha - m.atan2(ca - cb, d + sa - sb) + p / 2.)
        q = mod_to_pi(mod_to_pi(beta) - alpha - t + mod_to_pi(p))
```

```
    else:
        print('bad planner:', planner)

    path = [t, p, q]

    for i in [0, 2]:
        if planner[i].islower():
            path[i] = (2 * m.pi) - path[i]

    cost = sum(map(abs, path))

    return(path, mode, cost)

# calcaulte the length of a Dubin's path
def dubins_path_length(start, end, radius):
    (sx, sy, syaw) = start
    (ex, ey, eyaw) = end
    # convert the degree angle inputs to radians
    syaw = m.radians(syaw)
    eyaw = m.radians(eyaw)

    c = radius

    ex = ex - sx
    ey = ey - sy

    lex = m.cos(syaw) * ex + m.sin(syaw) * ey
    ley = - m.sin(syaw) * ex + m.cos(syaw) * ey
    leyaw = eyaw - syaw
    D = m.sqrt(lex ** 2.0 + ley ** 2.0)
    return D

# finds the Dubin's path between two points
def dubins_path(start, end, radius):
    (sx, sy, syaw) = start
    (ex, ey, eyaw) = end
    # convert the degree angle inputs to radians
    syaw = m.radians(syaw)
    eyaw = m.radians(eyaw)

    c = radius

    ex = ex - sx
    ey = ey - sy

    lex = m.cos(syaw) * ex + m.sin(syaw) * ey
```

```
    ley = - m.sin(syaw) * ex + m.cos(syaw) * ey
    leyaw = eyaw - syaw
    D = m.sqrt(lex ** 2.0 + ley ** 2.0)
    d = D / c

    # print the length of the path (distance of the points)
    print('D:', D)

    theta = mod_to_pi(m.atan2(ley, lex))
    alpha = mod_to_pi(- theta)
    beta = mod_to_pi(leyaw - theta)

    planners = ['LSL', 'RSR', 'LSR', 'RSL', 'RLR', 'LRL']

    bcost = float("inf")
    bt, bp, bq, bmode = None, None, None, None

    for planner in planners:
        solution = general_planner(planner, alpha, beta, d)

        if solution is None:
            continue

        (path, mode, cost) = solution
        (t, p, q) = path
        if bcost > cost:
            # best cost
            bt, bp, bq, bmode = t, p, q, mode
            bcost = cost

    print(bmode)
    return (list(zip(bmode, [bt*c, bp*c, bq*c], [c] * 3)))

##############################################################################
# Helpers
##############################################################################
# mormalizes an angle to the range [0, 2*pi)
def mod_to_pi(angle):
    return angle - 2.0 * m.pi * m.floor(angle / 2.0 / m.pi)

# constrains an angle to the range [-pi, pi]
def pi_to_pi(angle):
    while(angle >= m.pi):
        angle = angle - 2.0 * m.pi
    while(angle <= -m.pi):
        angle = angle + 2.0 * m.pi
```

```
    return angle

# create a linear path of points between two points
def linear(START, END, STEPS):
    """
    Creates a list of points on lines between a given start point and end poin
    start/end: [x, y, angle], the start/end point with given jaw angle.
    """
    x = []
    y = []
    Dx = END[0] - START[0]
    Dy = END[1] - START[1]
    dx = Dx/STEPS
    dy = Dy/STEPS
    for step in range(0, STEPS+1):
        x.append(step*dx + START[0])
        y.append(step*dy + START[1])
    return x, y

def arrow_orientation(ANGLE):
    # Returns x, y setoffs for a given angle to orient the arrows
    # marking the yaw of the start and end points.
    alpha_x = m.cos(m.radians(ANGLE))
    alpha_y = m.sin(m.radians(ANGLE))
    return alpha_x, alpha_y

# plot the shortest path between two points
def plot_trajectory(START, END, ARROWLENGTH=1):
    # plot the direct, linear connection between start and end
    x, y = linear(START, END, 1)
    plt.plot(x, y, "o:", color="Black", alpha=0.3)

    # find the values for the arrows at start and end given their angle
    sa_x, sa_y = arrow_orientation(START[2])
    ea_x, ea_y = arrow_orientation(END[2])

    # plot arrows with the orientations
    plt.arrow(START[0], START[1], sa_x, sa_y, width = 0.05, color="Black")
    plt.arrow(END[0], END[1], ea_x, ea_y, width = 0.05, color="Black")

    # statement
    print("Plotted trajectory for ", START, " --> ", END, ".")

# plot an arc in Matplotlib
def plot_arc(ax, center, radius, start_angle, end_angle, color, show_work):
    # Function to plot an arc on the given axis
```

```
    if show_work == 1:
        ax.add_artist(patches.Arc(
            xy=center,
            width=radius * 2,
            height=radius * 2,
            color=color,
            linewidth=1,
            ls="--"
        ))
    elif show_work == 2:
        ax.add_artist(patches.Arc(
            xy=center,
            width=radius * 2,
            height=radius * 2,
            angle=0,
            theta1=m.radians(start_angle),
            theta2=m.radians(end_angle),
            color=color,
            linewidth=1,
            ls="--"
        ))

# plot an arc in Matplotlib
def plot_ArcPath(ax, center, radius, start_angle, end_angle, color="Crimson"):
    # Create an arc with no fill to only show the outer border
    arc = patches.Arc(center, radius * 2, radius * 2, angle=0, theta1=start_ang
            color=color, linewidth=2, fill=False, linestyle="-")

    # Add the arc to the axis
    ax.add_artist(arc)

# plot a given Dubin's path in Matplotlib
def plot_solution(start, end, solution):
    """
    Plot the trajectory of a given solution for a Dubin's path between
    two points and show points along the steps of the path.
    """
    for show_work in range(3):
        # initialize the figure
        ax = plt.gca()
        # define a few important points
        radius = solution[0][2]
        current_position = start
        (sx, sy, syaw) = start
        (ex, ey, eyaw) = end
        ex = ex - sx
```

```
        ey = ey - sy

        # plot the S, L, or R elements from the solution
        for (mode, length, radius) in solution:
            #print("Current position: ", current_position)
            if mode == 'L':
                center = (
                    current_position[0] + m.cos(m.radians(current_position[2] -
                    current_position[1] + m.sin(m.radians(current_position[2] -
                )
                new_position = (
                    center[0] + m.cos(m.radians(current_position[2] - 90 + (180
                    center[1] + m.sin(m.radians(current_position[2] - 90 + (180
                    current_position[2] + (180 * length / (m.pi * radius))
                )
                #plotArc(ax, center, radius, current_position[2], current_posi
                plot_ArcPath(ax, center, radius, current_position[2]-90, new_po
            elif mode == 'R':
                center = (
                    current_position[0] + m.cos(m.radians(current_position[2]
                    current_position[1] + m.sin(m.radians(current_position[2]
                )
                new_position = (
                    center[0] + m.cos(m.radians(current_position[2] + 90 - (180
                    center[1] + m.sin(m.radians(current_position[2] + 90 - (180
                    current_position[2] - (180 * length / (m.pi * radius))
                )
                #plotArc(ax, center, radius, current_position[2] - m.pi, curren
                plot_ArcPath(ax, center, radius, new_position[2]+90, current_po
            elif mode == 'S':
                new_position = (
                    current_position[0] + m.cos(m.radians(current_position[2])
                    current_position[1] + m.sin(m.radians(current_position[2])
                    current_position[2],
                )
                if show_work == 2:
                    plt.plot((current_position[0], new_position[0]), (current_p
            else:
                print("Something went wrong.")

            current_position = new_position

# generate a Nazca cell for a given Dubin's path solution
def gds_solution(xs, pin1, pin2, solution):
    """
    Analogously to plotSolution() we draw the trajectory of a given solution fo
```

```
    two points, here, the pins, to be rendered in .gds!
    """
    # get the pin vectors from nazca pin1 and pin2
    start = pin1.xya()
    end = pin2.xya()

    # change the angle for the second pin
    new_end = list(end)
    new_end[2] = new_end[2] - 180
    end = tuple(new_end)

    # define the xs for the .gds file
    ic = Interconnect(xs=xs)

    # create the cell object for the path
    with nd.Cell("dubins-path") as C:
        #for show_work in range(3):

        # define important points
        radius = solution[0][2]
        current_position = start
        (sx, sy, syaw) = start
        (ex, ey, eyaw) = end
        ex = ex - sx
        ey = ey - sy

        # draw the S, L, or R elements from the solution
        for (mode, length, radius) in solution:
            if mode == 'L':
                center = (
                    current_position[0] + m.cos(m.radians(current_position[2] -
                    current_position[1] + m.sin(m.radians(current_position[2] -
                )
                new_position = (
                    center[0] + m.cos(m.radians(current_position[2] - 90 + (180
                    center[1] + m.sin(m.radians(current_position[2] - 90 + (180
                    current_position[2] + (180 * length / (m.pi * radius))
                )
                arc_angle = (180 * length / (m.pi * radius))
                ic.bend(radius=radius, angle=arc_angle).put()
            elif mode == 'R':
                center = (
                    current_position[0] + m.cos(m.radians(current_position[2]
                    current_position[1] + m.sin(m.radians(current_position[2]
                )
                new_position = (
```

```
                    center[0] + m.cos(m.radians(current_position[2] + 90 - (180
                    center[1] + m.sin(m.radians(current_position[2] + 90 - (180
                    current_position[2] - (180 * length / (m.pi * radius))
                )
                arc_angle = - (180 * length / (m.pi * radius))
                ic.bend(radius=radius, angle=arc_angle).put()
            elif mode == 'S':
                new_position = (
                    current_position[0] + m.cos(m.radians(current_position[2])
                    current_position[1] + m.sin(m.radians(current_position[2])
                    current_position[2],
                )
                xl = current_position[0] - new_position[0]
                yl = current_position[1] - new_position[1]
                l = m.sqrt(xl**2 + yl**2)
                ic.strt(length=l).put()
            else:
                print("Something went wrong.")

            current_position = new_position
    return C

# plot Dubin's path in Matplotlib
def plot_dubin(START, END, BENDRAD=1):
    # find the Dubin's path between START and END
    path = dubins_path(start=START, end=END, radius=BENDRAD)
    # plot the solution for the Dubin's path
    plot_solution(start=START, end=END, solution=path)
    # plot the start and end arrows
    #plot_trajectory(START, END)

def dubin_path_points(start, end, solution, res=0.1):
    """
    Given any solution to a Dubin's path, this function returns points
    as rastered along the path given a specified resolution.
    It is meant to be used to check if points of the path intersect with
    obstacles. It functions similarly to plot_solution() but returns
    the x and y coordinates for points of the rastered path.

    PARAMETERS
    ==========
    path:   The solution to a Dubin's path.
    res:    The resolution of the rastering.
    """
    # define important points
    radius = solution[0][2]
```

```
    current_position = start
    (sx, sy, syaw) = start
    (ex, ey, eyaw) = end
    ex = ex - sx
    ey = ey - sy

    # create empty lists for the x and y coordinates
    x = []
    y = []

    # draw the S, L, or R elements from the solution
    for (mode, length, radius) in solution:
        if mode == 'L':
            center = (
                current_position[0] + m.cos(m.radians(current_position[2] + 90
                current_position[1] + m.sin(m.radians(current_position[2] + 90
            )
            new_position = (
                center[0] + m.cos(m.radians(current_position[2] - 90 + (180 * 
                center[1] + m.sin(m.radians(current_position[2] - 90 + (180 * 
                current_position[2] + (180 * length / (m.pi * radius))
            )
            # raster the arc
            arc_angle = (180 * length / (m.pi * radius))
            for i in range(0, int(arc_angle/res)):
                x.append(center[0] + m.cos(m.radians(current_position[2] - 90 -
                y.append(center[1] + m.sin(m.radians(current_position[2] - 90 -
        elif mode == 'R':
            center = (
                current_position[0] + m.cos(m.radians(current_position[2] - 90
                current_position[1] + m.sin(m.radians(current_position[2] - 90
            )
            new_position = (
                center[0] + m.cos(m.radians(current_position[2] + 90 - (180 * 
                center[1] + m.sin(m.radians(current_position[2] + 90 - (180 * 
                current_position[2] - (180 * length / (m.pi * radius))
            )
            # raster the arc
            arc_angle = - (180 * length / (m.pi * radius))
            for i in range(0, int(arc_angle/res)):
                x.append(center[0] + m.cos(m.radians(current_position[2] + 90
                y.append(center[1] + m.sin(m.radians(current_position[2] + 90
        elif mode == 'S':
            new_position = (
                current_position[0] + m.cos(m.radians(current_position[2])) * 
                current_position[1] + m.sin(m.radians(current_position[2])) * 
```

```
                current_position[2]
                )
            # raster the straight line
            xl = current_position[0] - new_position[0]
            yl = current_position[1] - new_position[1]
            l = m.sqrt(xl**2 + yl**2)
            for i in range(0, int(l/res)):
                x.append(current_position[0] + m.cos(m.radians(current_positio
                y.append(current_position[1] + m.sin(m.radians(current_positio
        else:
            print("Something went wrong.")

        current_position = new_position

    return x, y
```

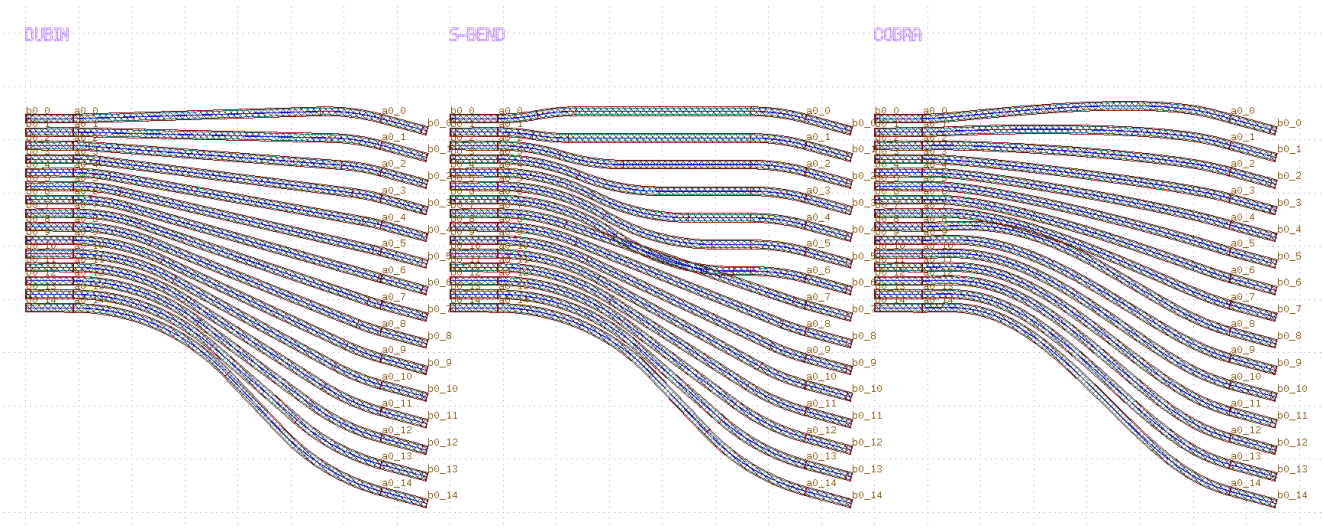
Finally, we can wrap our method into a single, simple to use function just like any other provided by the design library you might be using. In this case, I have been using the Nazca library[5] which comes with several interconnects, including straights and circular arcs which are used often but very tedious and slow to work with alone. Using `dubin_p2p()` as shown below, we can simply define the start pin, the end pin, and our code will route a Dubins path between them using the straights and circular arcs provided by Nazca. Of course, this can be easily adapted to other tools like GDSFactory[6].

```
##########################################################################
# Use this
##########################################################################

# create Dubins path between two pins in Nazca
def dubin_p2p(xs, pin1, pin2, radius=1500, width=3.2):
    """
    Finds and creates the shortest possible path between two vectors
    (pin1 and pin2) with a minimum bending radius,
    a so called "Dubin's path". This Dubins path is made of two
    circular bends and a straight waveguide.
    Returns a cell containing these waveguides.

    IMPORTANT
    =========
    In this version, you NEED to specify to put the path at the starting
    pin so if pin1=IO.pin["a0"] you must add .put(IO.pin["a0"]).
    Else, the Dubin's path will be generated correctly
    but possibly at the wrong position.

    PARAMETERS
```

```
    ==========
    xs:     Crosssection parameters.
    pin1:   The start pin to which the Dubins path attaches.
    pin2:   The end pin to where the Dubins path ends.
    radius: The minimum bending radius for the Dubins paths.
    width:  The width of the waveguides dubin_p2p creates.
    """
    # get the pin vectors from pin1 and pin2
    START = pin1.xya()
    END = pin2.xya()

    # change the angle for the second pin
    new_end = list(END)
    new_end[2] = new_end[2] - 180
    END = tuple(new_end)

    # find the Dubin's path between pin1 and pin2
    path = dubins_path(start=START, end=END, radius=radius)

    # create the Dubin's path with nazca using bends and straights
    return gds_solution(xs, pin1, pin2, solution=path)
```

6 Conclusion

There are also s-bends as well as, for example, cobra splines which are used all the time since they are much easier to use! In the figure below, a comparison between a dense array of Dubins paths and such arrays using Nazcas s-bends and cobra splines are shown:



As one can see, not only do other interconnects lead to over-bending of the waveguides and thus a longer path and greater losses, they are also less reliable, predictable, often break the design rules to not violate the minimum bending radius as is indeed the case here, and even intersect each other! Meanwhile, the Dubins paths behave extremely predictably. They clearly show the shortest path without unnecessary bends and they do not intersect each

other which allows for much denser layouts than would be possible with the other interconnects.

Again, the hours of headaches I personally avoided just by using Dubins paths are insane. It is also simply much more enjoyable to use.

Anyway, that's my little tip for those working on photonic integrated circuit layouts. I hope it helps!

Citing

If you found this work helpful for your own projects or research, please cite:

```
@article{
    author = {Quentin Wach},
    title = {Dubins Paths for Waveguide Routing},
    year = {2023}
}
```

References

1. Dubins, L. E., *"On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents"*. American Journal of Mathematics. 79 (3): 497–516, 1957 ↩

2. Wikipedia: Dubins Paths (Accessed: Feb 20, 2024) ↩

3. David A. Anisi, *"Optimal Motion Control of a Ground Vehicle"*, 2003 ↩

4. Atsushi Sakai, *"Python Robotics: Dubins Path Planning"*, GitHub. (Accessed: Feb 20, 2024) ↩

5. Nazca Design: Photonic IC Design Framework ↩

6. GDSFactory, GitHub ↩